
Seguimiento

Trabajo Práctico Final

Algoritmos y Estructuras de Datos

- Antes de empezar a trabajar en el código, ya nos habíamos decidido a empezar a implementar el método **BuscarConOrden**, ya que teníamos mayor experiencia con estos tipos de algoritmos y, a partir de la finalización de este, podríamos saber que tener en cuenta para los demás.

Con respecto a los algoritmos usados, optamos por comenzar con una **Búsqueda Secuencial** y luego con un **Bubble Sort**, no porque sean los óptimos, sino porque ambas tienen una complejidad $O(N^2)$, lo que haría que se note aún más la diferencia al momento de realizar la “consulta 1”, además de ser más simples de implementar.

- Al momento de implementar **BuscarConOrden**, el único problema que encontramos al momento de llenar la lista **Collected**, ya que el conteo de cada elemento era correcto, pero se agregaba varias veces a la lista.

-Ejemplo:

List<string>:

rojo

azul

rojo

----->

List<dato>:

rojo, 2 ocurrencias

azul, 1 ocurrencia

rojo, 1 ocurrencia

rojo termina apareciendo 2 veces, creando un dato innecesario.

- Para resolverlo, se creó “**bool existe = false**” al principio, para revisar si el dato ya existe dentro de la lista, realizando un **break** si ya existe y un **continue** si no existe, agregándolo a la lista.
- Luego, nos dimos cuenta de que no usamos el parámetro “**cantidad**” (cantidad de resultados que se quieren ver). Nuestra solución fue usar “**remove range**” y borrar todo lo que este delante de los resultados que queremos ver.
- Una manera de mejora esta parte del código sería que cargue los datos más repetidos al **collected**, pero para eso habría que reestructurar todo el código...

//ver como ordenar lo de abajo

- Tuvimos problemas al hacer el **Heap Sort**, ya que los únicos que hicimos recibían arreglos de enteros. No estamos acostumbrados a usar genéricos, pero es la mejor manera de implementarlo.
- Se agrego una variable **bool maxHeap** dentro de la clase **heap** donde se puede elegir si se ordena de forma ascendente o descendente (**maxHeap** o **minHeap**).
- La idea es optimizar la cantidad de operaciones para que se note la diferencia en la complejidad al hacer la **consulta 1**. la solución que encontramos fue usar la estructura **Dictionary**:

```
dictionary<string, int> contador = new dictionary <string, int>
```

Complejidad $O(N)$, a diferencia de un $O(N^2)$ de la otra busqueda.

- Creamos una clase **Heap** y se la puso en estructura.
- Hubo un problema al implementar **dictionary**, ya que la clase **heap** usa arreglos, mientras que los métodos en la clase **estrategia** utilizan listas. Eso se puede solucionar convirtiendo el **collected** y los datos a un **array**, para luego, convertir de nuevo la lista.
- Al armar el **heap** se usaron genéricos comparables, pero no hicimos que la clase **dato** sea comparable, así que se le agrego:

```
public class dato: Icomparable<dato> {
    .
    .
    .
    Public int compareTo(dato otro);
    return ocurrencia.CompareTo(otro.ocurrencia); }
```

- Para la **consulta 1** descubrimos que existe la estructura **StopWatch**, que sirve como un cronometro. La idea sería:

1) iniciar cronometro.	<code>StopWatch sw = new StopWatch();</code>
2) ejecutar método.	<code>StopWatch.start;</code>
3) parar cronometro.	<code>buscar con heap(...);</code>
4) guardar tiempo.	<code>tiempo = sw.elapsedmilliseconds;</code>

BuscarConHeap debería ser más rápido ya que su complejidad está compuesta por:

Dictionary $O(N)$

$$+ \\ \textit{HeapSort } O(n \log n)$$

Mientras que **BuscarConOtro** está compuesto por:

BubbleSort

$$+ \\ \textit{Doble for} = O(n^2)$$

Aunque, esta diferencia de tiempo debería ser más notoria mientras más grande sea el archivo.

En nuestro caso, usando un archivo pequeño dentro del mismo proyecto, siempre veíamos mínimo una diferencia de 1ms.